FPT UNIVERSITY

MÔN HỌC EMBEDDED LINUX SYSTEMS

BÁO CÁO ASSIGNMENT 02

Sensor Monitoring System

Thiết kế và kiểm chứng ứng dụng giám sát cảm biến đa luồng trên Linux ARM

Sinh viên Phạm Trần Gia Long

Mã số sinh viên SE201095

Nền tảng Linux 5.15 ARMv7 trong QEMU

Ngày hoàn thiện báo cáo 22 tháng 10 năm 2026

Hệ thống đọc dữ liệu cảm biến mô phỏng, tính toán thống kê, phát cảnh báo và ghi log. Báo cáo trình bày thiết kế, cơ chế đồng bộ, kết quả kiểm tra bộ nhớ và phân tích độ trễ thức dậy.

Tóm tắt và nội dung báo cáo

Hệ thống SMS gồm một kernel module và ứng dụng bốn worker thread. Driver cung cấp /dev/sms_sensor và /proc/sms_stats; ứng dụng đọc theo chu kỳ 500 ms, xử lý qua circular buffer, phát cảnh báo bằng POSIX message queue và ghi log bằng một Logger riêng.

Bản cuối trên QEMU chạy 60,030 giây, đọc và xử lý 119 mẫu, không tràn buffer và không có lỗi nghiêm trọng. Valgrind trên bản HOST ghi nhận 20 lần cấp phát, 20 lần giải phóng, 0 byte tại exit và 0 lỗi. VmRSS và VmSize giữ nguyên tại ba thời điểm quan sát.

Hai phiên jitter dài thu được 1.020 mẫu cho mỗi chính sách. FIFO có P99 1.325 µs, thấp hơn OTHER 2.092 µs khoảng 36,66%; mean của FIFO lại hơi cao hơn. Kết quả cho thấy cải thiện ở phần đuôi phân bố trong cặp phiên đo này, chưa chứng minh bảo đảm thời gian thực cứng.

Cách đọc các kết quả

Các kiểm tra HOST dùng dữ liệu MOCK trong userspace. Các kiểm tra QEMU dùng DRIVER. Hai phiên jitter dài và các phiên strace/perf được thực hiện trước đợt rà soát cuối. GDB, Valgrind và phiên ARM 60 giây trong logs/final_review là kiểm tra sau sửa. Sự phân biệt này giúp đối chiếu đúng bản mã và điều kiện thực nghiệm.

Phần	Nội dung
1 đến 3	Phạm vi, môi trường, kiến trúc và vòng đời
4 đến 7	Driver, Sensor, Processor, IPC và Logger
8 đến 9	Đồng bộ, thứ tự khóa và dừng hệ thống
10 đến 15	Kiểm tra chức năng, bộ nhớ, GDB, strace và perf
16 đến 17	Jitter và đề xuất thời gian thực
18 đến 19	Reflection, tái hiện và tài liệu tham khảo

1 Phạm vi và môi trường thực nghiệm

Bài toán là giám sát cảm biến công nghiệp ở mức mô phỏng phần mềm. Nhiệt độ và độ ẩm không đo từ môi trường vật lý: kernel driver sinh số ngẫu nhiên khi có lần đọc hợp lệ. Các phép đo thời gian, bộ nhớ và hành vi thread là quan sát thực tế của chương trình khi chạy.

Thành phần	Cấu hình đã sử dụng
HOST	Ubuntu x86_64; kernel 7.0.0-38-generic trong phiên perf
Target	QEMU vexpress-a9, Cortex-A9, 512 MiB RAM, 2 vCPU
Kernel target	Linux 5.15.0, ARMv7
Toolchain	arm-none-linux-gnueabihf-gcc 13.2.1
Userspace	BusyBox initramfs; ứng dụng ARM liên kết động
Thư viện ARM	libc.so.6, libm.so.6, ld-linux-armhf.so.3 từ cùng toolchain
Chức năng kernel	MODULES, PROC_FS, POSIX_MQUEUE, HIGH_RES_TIMERS được bật
Công cụ kiểm tra	GDB multiarch, Valgrind 3.26.0, strace, perf 7.0.14

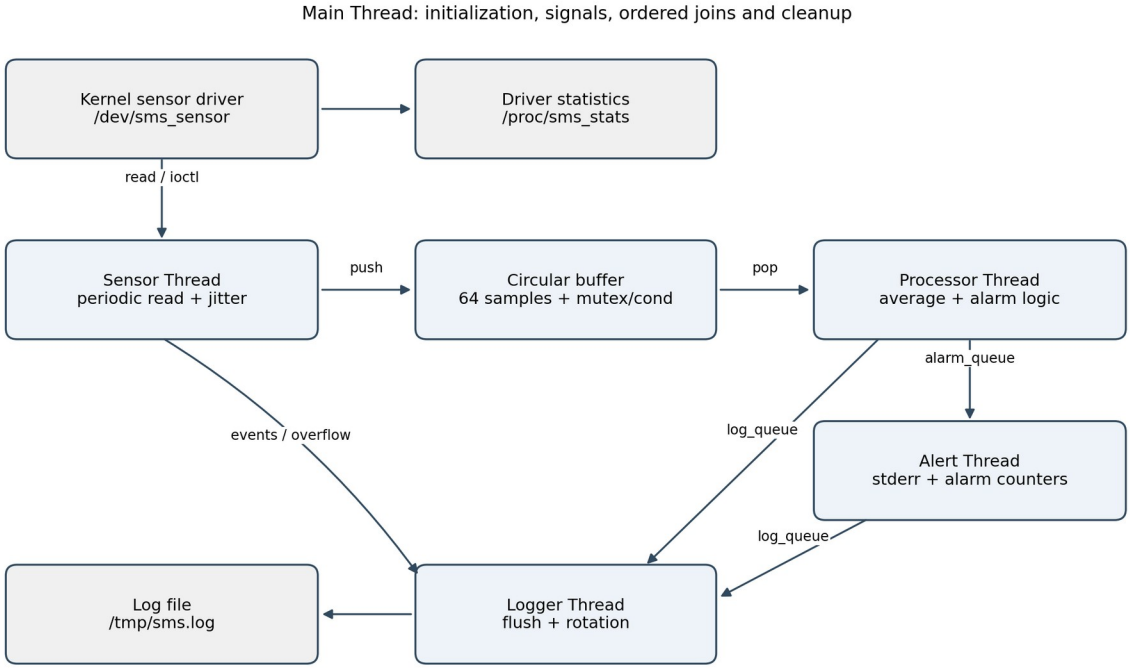
Kế thừa từ các bài trước

Lab 1 cung cấp kernel, device tree và môi trường boot. Lab 2 cùng Assignment 1 cung cấp cách tổ chức character device và ioctl. Lab 3 cung cấp phương pháp debug, kiểm tra bộ nhớ và profiling. Assignment 2 tích hợp các kỹ năng đó vào ứng dụng SMS của project này [1].

Kernel target và kernel HOST là hai môi trường khác nhau. Module ARM được build với source kernel target. Binary HOST phục vụ chẩn đoán; binary ARM được cài vào rootfs để chạy trong QEMU. Các số liệu HOST không được dùng thay cho số liệu driver ARM.

Minh chứng: output/kernel.config; logs/perf_environment.txt; các log QEMU và lệnh build đã lưu.

2 Kiến trúc hệ thống



Hình 1 Luồng dữ liệu và các cơ chế giao tiếp trong SMS

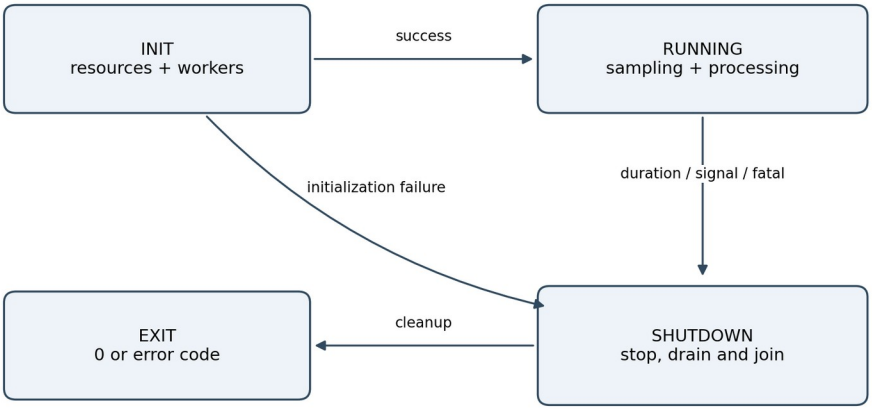
Sensor là producer duy nhất của shared_buffer; Processor là consumer duy nhất. Processor cập nhật cửa sổ tính trung bình riêng, gửi AlarmMessage cho Alert và gửi nội dung mẫu cho Logger. Alert tăng bộ đếm theo loại rồi chuyển nội dung cảnh báo sang Logger.

Buffer là mảng 64 phần tử cấp phát một lần. Cách này giữ chi phí lưu trữ có giới hạn, tránh cấp phát theo từng mẫu và giữ thứ tự dữ liệu dễ kiểm tra. Khi đầy, mẫu mới bị bỏ và overflow_count tăng; đây là lựa chọn không chờ chỗ trống cho Sensor.

Message queue phù hợp với thông điệp có kích thước xác định và có sẵn cơ chế chờ nhận. Mỗi thông điệp được sao chép vào queue, nên Alert không giữ con trỏ tới biến stack của Processor. Logger sở hữu file duy nhất, giúp các worker khác không phải tự quản lý timestamp, flush và rotation.

Minh chứng: app/sms_app.h, circular_buffer.c, processor_thread.c, alert_thread.c và logger_thread.c.

3 Vòng đời và quản lý tài nguyên



Hình 2 Các trạng thái của ứng dụng và điều kiện chuyển trạng thái

Ở INIT, Main đọc CLI, khởi tạo buffer, cấp phát mảng jitter và chặn SIGINT/SIGTERM trước khi tạo worker. Hai queue được mở với tên chứa PID và unlink ngay sau khi mở; descriptor vẫn được dùng đến khi đóng. Main tạo Logger, Alert, Processor rồi Sensor.

Ở RUNNING, Main chờ tín hiệu hoặc hết thời lượng. Sensor dùng FIFO priority 50 mặc định; các worker còn lại dùng chính sách thông thường. Trong DRIVER, mlockall phải thành công trước khi bắt đầu. MOCK trên HOST tắt khóa bộ nhớ để phục vụ công cụ chẩn đoán.

Ở SHUTDOWN bình thường, Main yêu cầu Sensor dừng, join Sensor và Processor, gửi ALARM_STOP rồi join Alert, cuối cùng gửi LOG_STOP và join Logger. Main xuất CSV jitter, in summary, đóng queue, giải phóng mảng và hủy buffer. Mã thoát 0 biểu thị hoàn tất bình thường.

Nếu khởi tạo một phần thất bại, created và các cờ trạng thái xác định tài nguyên nào đã tồn tại. Lỗi đầu tiên được giữ trong fatal_error. Nhánh lỗi ưu tiên dừng hệ thống và có thể không drain hết message; nhánh dừng bình thường mới bảo đảm thứ tự xử lý hết dữ liệu còn lại.

4 Kernel driver mô phỏng cảm biến

sms_sensor_driver đăng ký miscdevice với minor cấp động, tên sms_sensor và quyền 0600. read trả một dòng CSV gồm timestamp monotonic tính bằng ms, nhiệt độ và độ ẩm. Kernel tính bằng số nguyên phần mười để không cần phép toán dấu phẩy động.

```
temperature10 = 150 + get_random_u32() % 301;
humidity10    = 300 + get_random_u32() % 601;
CSV: timestamp_ms,temperature,humidity\n
```

Giao diện	Hành vi và lỗi chính
read	count=0 trả 0; buffer nhỏ hơn 64 byte bị từ chối; trả trọn một dòng CSV
Giới hạn chu kỳ	Chưa đủ interval từ lần đọc thành công trước: -EAGAIN
copy_to_user	Lỗi con trỏ: -EFAULT; không commit thống kê mẫu
SMS_SET_INTERVAL	Nhận __u32 qua _IOW; chấp nhận 1 đến 60000 ms
ioctl không hợp lệ	Mã lệnh lạ: -ENOTTY; interval ngoài miền: -EINVAL
/proc/sms_stats	read_count, has_sample, giá trị cuối, interval và uptime

Mutex sms_lock bảo vệ interval, thời điểm mẫu trước và thống kê. Driver chỉ tăng read_count và cập nhật giá trị cuối sau khi copy_to_user thành công. Đọc procfs lấy snapshot dưới cùng mutex và không tạo mẫu mới. Thiết kế này tránh tăng counter cho lần đọc chưa trả dữ liệu được.

Nếu tạo procfs thất bại sau misc_register, init hủy miscdevice trước khi trả lỗi. exit xóa procfs và miscdevice. Một lần nạp thành công chỉ cần insmod một lần; File exists khi nạp lại cho biết module đã tồn tại, không phải lỗi tạo dữ liệu.

Minh chứng: driver/sms_sensor_driver.c, sms_sensor.h; logs/driver_functional_session.txt.

5 Sensor và chu kỳ lấy mẫu

Sensor mở device_path bằng O_RDONLY | O_NONBLOCK và gọi SMS_SET_INTERVAL theo -i. Ứng dụng mặc định 500 ms và cấu hình driver về cùng khoảng này khi mở thiết bị. CSV được kiểm tra đủ trường, dấu phân cách, newline, miền giá trị và tính hữu hạn trước khi đưa vào buffer.

Lịch thức dậy tuyệt đối

```
expected = startup_monotonic + interval
wait_until(expected)
actual = clock_gettime(CLOCK_MONOTONIC)
jitter_us = timespec_diff_us(actual, expected)
read_and_push_sample()
expected += interval
```

sensor_wait_until dùng clock_nanosleep với TIMER_ABSTIME và CLOCK_MONOTONIC [3]. Khoảng ngủ dài được chia thành các mốc tối đa 100 ms để kiểm tra stop_requested. Sau xử lý, nếu đã lỡ các mốc tiếp theo, Sensor tăng skipped_periods thay vì chạy liên tục để bắt kịp lịch cũ.

Thử lại khi driver chưa sẵn sàng

EAGAIN được thử lại sau khoảng 1 ms, trong ngân sách nửa chu kỳ tính từ expected. EINTR có bộ đếm riêng. EOF hoặc CSV sai được ghi nhận và hoãn mẫu, lỗi đọc nghiêm trọng kích hoạt dừng. Driver được đóng trong nhánh out. Buffer được đóng và đánh thức consumer khi Sensor kết thúc.

Hai đồng hồ logic có thể lệch pha: lịch ứng dụng bám mốc tuyệt đối, còn driver tính khoảng tối thiểu từ lần đọc thành công gần nhất. Lần đọc trước bị chậm sẽ khiến lần thức dậy kế tiếp phải retry. Vì vậy jitter thức dậy thấp vẫn có thể đi kèm nhiều EAGAIN; phép đo jitter hiện tại không bao gồm thời gian thử lại thiết bị.

Minh chứng: app/sensor_thread.c; các trường sensor_eagain_retries và sensor_deferred_samples trong log.

6 Processor và tính đúng của cảnh báo

Processor chờ cb_pop khi buffer rỗng và tính rolling average trên tối đa 10 mẫu gần nhất. Khi cửa sổ đầy, mẫu cũ được trừ khỏi tổng trước khi thêm mẫu mới. Trạng thái cửa sổ chỉ do Processor truy cập nên không cần một mutex thứ hai.

Loại	Điều kiện
TEMP_HIGH	Nhiệt độ mẫu hiện tại > temperature_threshold, mặc định 40°C
HUMID_HIGH	Độ ẩm mẫu hiện tại > humidity_threshold, mặc định 80%
TEMP_SPIKE	Chênh lệch nhiệt độ tuyệt đối so với mẫu trước > 5°C

Cảnh báo dùng giá trị từng mẫu, không dùng rolling average. Mẫu đầu tiên chưa có mẫu trước nên không phát TEMP_SPIKE. Mỗi mẫu tạo tối đa ba AlarmMessage.

Sửa lỗi ở biên 5 độ C

Phép tính float32 20.2f - 15.2f cho khoảng 5.0000009537. So sánh trực tiếp > 5.0f có thể báo spike dù độ chênh danh nghĩa đúng 5°C. Bản cuối làm tròn hai nhiệt độ sang đơn vị 0,1°C trước khi trừ, phù hợp độ phân giải driver và MOCK.

```
./test_circular_buffer
PASS: full, overflow, wrap-around, close and drain
PASS: concurrent produced=10000 consumed=10000 remaining=0
=== CIRCULAR BUFFER TEST PASSED ===
./test_processor
PASS: 502 transitions of exactly 5.0 C do not raise TEMP_SPIKE
PASS: 500 transitions of 5.1 C raise TEMP_SPIKE
PASS: thresholds, first sample and rolling window after wrap-around
=== PROCESSOR TEST PASSED ===
```

Hình 3 Kiểm tra buffer và ngưỡng cảnh báo trên laptop

Regression test kiểm tra 502 chuyển tiếp đúng 5°C theo cả hai chiều và 500 chuyển tiếp 5,1°C. Bản cũ thất bại ở kiểm tra biên; bản sửa đạt. Nếu nguồn dữ liệu tương lai có độ phân giải cao hơn 0,1°C, quy tắc lượng tử hóa này cần được thiết kế lại.

7 Message queue Alert và Logger

alarm_queue chứa AlarmMessage; log_queue chứa LogMessage với tối đa 256 byte text. Mỗi queue có sức chứa 10 message. Tất cả message dùng priority 0 để tránh việc thông điệp STOP vượt các message dữ liệu trong cùng queue. Queue được tạo trước worker và descriptor được đóng sau khi join [4].

sms_send_message dùng mq_timedsend với deadline 2 giây. EINTR giữ nguyên deadline. Alert và Logger dùng mq_timedreceive với deadline 1 giây để vừa chờ dữ liệu vừa có cơ hội kiểm tra fatal_error. Đây vẫn là chờ blocking, không phải vòng lặp busy polling.

Xử lý cảnh báo

Alert kiểm tra kích thước và loại message, in timestamp monotonic của mẫu lên stderr, tăng bộ đếm theo loại rồi gửi một bản mô tả cho Logger. Main chỉ đọc các bộ đếm này sau khi Alert kết thúc. Định dạng tiêu chuẩn giúp đối chiếu các dòng cảnh báo với mẫu gốc.

Sở hữu và lưu file log

Logger là thread duy nhất mở, ghi, flush, rename và đóng file log. Mỗi dòng có ngày giờ lịch, millisecond và offset múi giờ. Các log QEMU dùng +0000; thời gian mẫu monotonic không được hiểu là giờ lịch. Flush theo CLOCK_MONOTONIC mỗi 5 giây, kể cả khi queue liên tục có dữ liệu, rồi flush thêm ở shutdown.

Giới hạn xoay file hiện dùng 1.048.576 byte, tức 1 MiB. Khi vượt giới hạn, Logger flush, đóng file, đổi thành path.1 và mở file mới. Chỉ giữ một bản cũ. Sensor gửi startup, shutdown, deferred và cảnh báo overflow qua Logger, nên các sự kiện đó có mặt trong file chứ không chỉ xuất hiện trên terminal.

Thử nghiệm HOST bổ sung khi rà soát xác nhận rotation với file mồi lớn hơn 1 MiB và flush định kỳ khi ứng dụng còn chạy. Đây là kiểm tra bổ sung trong môi trường rà soát; log ứng dụng do sinh viên chạy trên laptop vẫn là minh chứng chính của project.

8 Dữ liệu dùng chung và phòng ngừa race

Dữ liệu	Chủ ghi và cách bảo vệ
Buffer items, head, tail, size, closed	Sensor, Processor hoặc Main; cùng buffer.mutex
produced, consumed, overflow	Cập nhật và snapshot dưới buffer.mutex
stop_requested, fatal_error	Nhiều thread truy cập bằng C11 atomic
samples_read và thống kê đọc	Sensor ghi; Main đọc sau join Sensor
samples_processed	Processor ghi; Main đọc sau join Processor
Các bộ đếm alert	Alert ghi; Main đọc sau join Alert
Mảng jitter và jitter_count	Sensor ghi; Main xuất sau join
Cấu hình và descriptor queue	Main khởi tạo trước pthread_create; không sửa khi worker chạy
Rolling window và file log	Lần lượt do Processor và Logger sở hữu riêng
Thống kê kernel driver	sms_lock bảo vệ read, ioctl và procfs

Tình huống race tiềm ẩn được xử lý trong thiết kế

Nếu Main đọc samples_read, jitter_count hoặc bộ đếm Alert khi worker vẫn cập nhật, việc dùng integer thông thường không tạo đồng bộ hợp lệ. Thay vì thêm khóa cho mọi bộ đếm, thiết kế đặt print_summary và xuất jitter sau các pthread_join tương ứng. Trong RUNNING, Main chỉ quan sát các cờ atomic. Đây là biện pháp loại bỏ truy cập đồng thời không đồng bộ tại điểm tổng kết.

Một nguy cơ khác là kiểm tra size bên ngoài mutex rồi mới chờ condition: producer có thể signal giữa hai thao tác và consumer bỏ lỡ tín hiệu. cb_pop kiểm tra điều kiện trong vòng while dưới cùng mutex; pthread_cond_wait nhả khóa và chờ theo giao thức của condition variable [2]. Khi thức dậy, điều kiện được kiểm tra lại.

Các tình huống trên là phân tích race tiềm ẩn từ luồng truy cập. GDB và các lần chạy ổn định không tự chứng minh đã tìm thấy một race thực tế, và Valgrind Memcheck không thay thế công cụ chuyên kiểm tra race.

9 Phân tích chờ khóa và dừng sạch

Ở userspace, chỉ có một mutex tường minh cho shared_buffer. cb_try_push, cb_pop, cb_close và cb_get_stats lấy khóa này rồi trả khóa trong chính hàm. Không có đường gọi giữ buffer.mutex rồi chờ gửi queue, ghi file hoặc pthread_join. Khi chờ dữ liệu, pthread_cond_wait nhả mutex để Sensor có thể đẩy mẫu [2].

Đoạn xử lý	Khóa hoặc chờ đang giữ
Push hoặc pop buffer	Chỉ buffer.mutex; đoạn cập nhật ngắn
Gửi alarm hoặc log	Buffer.mutex đã được nhả; queue có timeout
Ghi và xoay log	Logger sở hữu FILE; không lấy buffer.mutex
Main join worker	Không giữ buffer.mutex
Driver read hoặc procfs	Chỉ sms_lock trong kernel; không giữ mutex userspace

Không hình thành chu trình acquire giữa các mutex trong các đường xử lý này. Điều đó giải quyết deadlock do thứ tự khóa ở mức ứng dụng. Kết luận dựa trên giả thiết các primitive hệ thống và I/O trả về; nó không tạo giới hạn worst-case cho mọi lỗi filesystem hoặc cho độ trễ lập lịch.

Thứ tự dừng bảo toàn dữ liệu

Đóng buffer chỉ ngăn push mới; Processor vẫn pop hết dữ liệu còn lại. ALARM_STOP được gửi sau khi Processor đã kết thúc, LOG_STOP sau khi mọi thread tạo log đã kết thúc. Các message cùng priority giúp STOP nằm sau dữ liệu đã được đưa vào queue. Nhánh fatal có timeout và kiểm tra cờ để tránh chờ producer đã chết.

Phiên SIGINT dưới Valgrind của bản cuối ghi rõ received signal 2, 69 mẫu đọc và 69 mẫu xử lý, buffer_remaining=0, các worker dừng, Logger flush và exit_code=0. Đây là bằng chứng dừng sạch cho workload đã thử; không phải kiểm thử mọi tình huống lỗi.

Minh chứng: logs/final_review/valgrind_app_output.txt và valgrind_exit_code.txt.

10 Kết quả chức năng trên ARM

Chỉ số bản cuối	Giá trị
Chế độ và chu kỳ	DRIVER, FIFO priority 50, 500 ms
Thời gian và số mẫu	60030 ms; 119 đọc, 119 xử lý
Cảnh báo theo loại	TEMP_HIGH 25; HUMID_HIGH 17; TEMP_SPIKE 77
Overflow và lỗi nghiêm trọng	0 và 0; exit_code=0

```
=== FINAL ARM RESULT ===
APP_EXIT_CODE=0
[MAIN] mlockall OK
[MAIN] mode=DRIVER policy=FIFO interval=500 ms duration=60 s log=/tmp/sms_final.log
[SENSOR] policy=FIFO priority=50 mode=DRIVER
[ALERT][18770] TYPE: TEMP_HIGH VALUE: 40.7 THRESHOLD: 40.0
[ALERT][18770] TYPE: TEMP_SPIKE VALUE: 21.7 THRESHOLD: 5.0
[ALERT][76814] TYPE: TEMP_SPIKE VALUE: 13.9 THRESHOLD: 5.0
[MAIN] duration reached
[SENSOR] stopped: samples=119 skipped_periods=0
[PROCESSOR] stopped: processed=119
[ALERT] stopped: TEMP_HIGH=25 HUMID_HIGH=17 TEMP_SPIKE=77
[LOGGER] stopped
[JITTER] mode=DRIVER policy=FIFO interval_ms=500 samples=119 file=/tmp/sms_final.log.jitter.csv
[MAIN] exit_code=0

=== SMS SUMMARY ===
mode=DRIVER policy=FIFO interval_ms=500
uptime_ms=60030
samples_read=119
sensor_read_errors=0
sensor_eagain_retries=1709
sensor_interrupted_reads=0
sensor_deferred_samples=0
samples_processed=119
total_produced=119
total_consumed=119
overflow_count=0
buffer_remaining=0
TEMP_HIGH=25
HUMID_HIGH=17
TEMP_SPIKE=77
jitter_samples=119 capacity=100000
fatal_error=0
=== SMS END ===
```

Hình 4 Kết quả phiên QEMU ARM sau đợt sửa cuối

Driver tăng read_count từ 0 lên 119, khớp số mẫu ứng dụng. sensor_read_errors, sensor_deferred_samples và skipped_periods đều bằng 0. Có 1709 lượt EAGAIN retry. 119 mẫu nằm trong miền yêu cầu 120 ±10; mốc dừng Main và mốc bắt đầu chu kỳ Sensor khác nhau nên không buộc chính xác 120 mẫu ở ranh giới 60 giây.

Minh chứng: logs/final_review/qemu/sms_final_output.txt; sms_final_stats_before.txt; sms_final_stats_after.txt.

11 Kiểm tra bộ nhớ bằng Valgrind

Bản HOST/MOCK chạy với --other, chu kỳ 500 ms và -d 0 dưới Memcheck. Bộ điều khiển chờ 35 giây từ lúc tạo process, sau đó gửi SIGINT và chờ chương trình kết thúc. Ứng dụng ghi uptime 34685 ms sau phần khởi động của công cụ. Không có early exit hoặc kill cưỡng bức.

```
==79028==
==79028==
==79028== HEAP SUMMARY:
==79028==     in use at exit: 0 bytes in 0 blocks
==79028==   total heap usage: 20 allocs, 20 frees, 820,186 bytes allocated
==79028==
==79028== All heap blocks were freed -- no leaks are possible
==79028==
==79028== For lists of detected and suppressed errors, rerun with: -s
==79028== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Hình 5 Heap summary và error summary của Valgrind bản cuối

Kết quả	Giá trị
Bộ nhớ còn cấp phát tại exit	0 byte trong 0 block
Tổng heap usage	20 allocs, 20 frees, 820186 byte allocated
Error summary	0 errors, 0 contexts, suppressed 0
Mã thoát Valgrind và ứng dụng	0 và 0
Mẫu đọc và xử lý	69 và 69
TEMP_HIGH / HUMID_HIGH / TEMP_SPIKE	11 / 10 / 68

Buffer và mảng jitter được cấp phát trước khi worker chạy, giải phóng sau join. Thread không malloc một SensorData cho từng mẫu. Tổng alloc Valgrind vẫn lớn hơn hai vì libc, pthread, stdio và thao tác xuất CSV có thể dùng heap. 820186 byte là tổng lượng cấp phát trong phiên, không phải RSS hoặc mức dùng heap đồng thời cao nhất.

Không còn block tại exit cũng loại trừ definitely lost, possibly lost và still reachable trong phiên này. Kết quả hỗ trợ tính đúng về bộ nhớ trên đường thực thi đã thử; cần giữ phân biệt với kiểm tra lỗi trên kernel module hoặc trên phần cứng thực.

Minh chứng: logs/final_review/valgrind_full.txt; valgrind_exit_code.txt; valgrind_app_output.txt.

12 Footprint và bản đồ bộ nhớ

Mốc	VmSize kB	VmRSS kB	VmLck kB	Threads
5 giây	4632	3480	4620	5
30 giây	4632	3480	4620	5
55 giây	4632	3480	4620	5

Ba snapshot thuộc PID 66 của bản ARM cuối. VmSize và VmRSS không tăng giữa các mốc; Threads=5 tương ứng Main cộng bốn worker. Đây là quan sát ổn định trong cửa sổ thử 60 giây, không suy rộng thành chứng minh không leak cho thời gian chạy vô hạn.

Vùng trong maps	Giải thích
00010000–00015000 r-xp	Text của /usr/bin/sms_app, chứa mã thực thi
00016000–00017000 rw-p	Vùng dữ liệu ghi được của executable
00017000–00038000 [heap]	Heap do allocator quản lý; không bao gồm mọi mmap cấp phát lớn
76be9000–76c29000 rw-p	Anonymous mapping 256 KiB, phù hợp cấu hình stack worker
76c2a000–76c6a000 rw-p	Một mapping 256 KiB khác cạnh guard page
76db0000–76eb2000 r-xp	Mã libc.so.6 dùng cho runtime và syscall wrappers
76ebf000–76eff000 r-xp	Mã libm.so.6 dùng cho toán học
76f03000–76f1f000 r-xp	Dynamic loader ld-linux-armhf.so.3
7eed4000–7eef5000 [stack]	Stack của Main, phân biệt với các stack worker anonymous
7eefa000–7eefb000 [vdso]	Vùng ánh xạ hỗ trợ một số dịch vụ kernel trong userspace

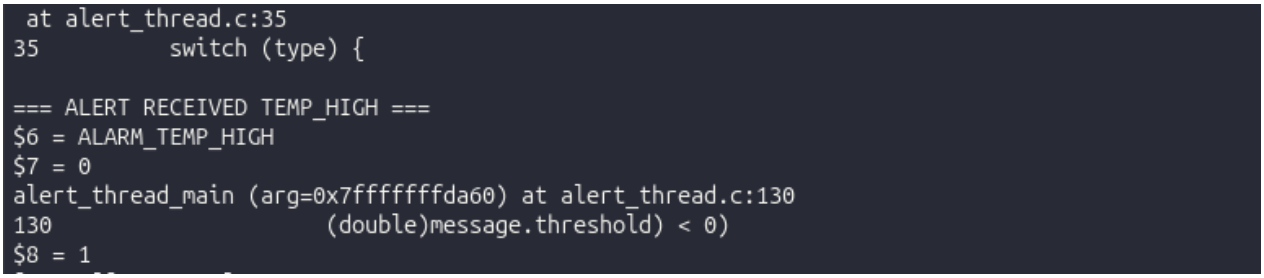
Bốn worker được đặt stack 256 KiB. Maps có các vùng anonymous và guard page phù hợp, nhưng một vùng có thể gộp với cấp phát khác. Không thể gán chắc chắn tên Sensor, Processor, Alert hay Logger cho từng mapping chỉ từ snapshot này. Cần địa chỉ stack của từng thread để đối chiếu cụ thể; mảng jitter 100000 phần tử int64_t cũng chiếm 800000 byte và có thể được allocator cấp bằng mmap.

Minh chứng: logs/final_review/qemu/sms_final_memory_*s.txt và sms_final_maps_5s.txt.

13 GDB kiểm chứng đường đi cảnh báo

Phiên GDB dùng sms_app_debug build -O0 -g3 trên HOST/MOCK. Kịch bản B đặt breakpoint ở processor_analyze, đổi nhiệt độ mẫu đầu từ 15°C thành 45°C rồi quan sát kết quả và điểm nhận message trong Alert. Scheduler locking được bật khi step một thread, sau đó tắt để các worker tiếp tục giao tiếp.

```
sample trước sửa: temperature=15, humidity=30
set variable sample->temperature = 45.0
result.average_temperature = 45
result.alarm_count = 1
result.alarms[0] = TEMP_HIGH, value=45, threshold=40
shared_buffer: head=1, tail=1, size=0
```



Hình 6 Alert nhận TEMP_HIGH và bộ đếm chuyển từ 0 sang 1

Thread trong phiên	Trạng thái quan sát qua backtrace
Main	sigtimedwait trong wait_for_stop
Logger	mq_timedreceive trên log_queue
Alert	mq_timedreceive, sau đó vào alert_count
Processor	processor_thread_main sau processor_analyze
Sensor	clock_nanosleep trong sensor_wait_until

Transcript chứa info threads, thread apply all bt và breakpoint tại alert_count với type=ALARM_TEMP_HIGH. Bộ đếm tăng 0 lên 1, xác nhận message đã tới consumer. Cuối phiên, sáu mẫu được xử lý và inferior exited normally. Các giá trị này thuộc phiên có can thiệp debugger, không dùng làm thống kê cảm biến ngẫu nhiên bình thường.

Minh chứng: logs/final_review/gdb_session.txt và gdb_sms.log.

14 Phân tích system call bằng strace

Phiên strace theo dõi mọi thread bằng -f, thu bảng tổng hợp -c trong 30 giây trên HOST/MOCK/OTHER. Ứng dụng xử lý 60 mẫu và thoát 0. Bảng dưới chọn năm syscall được gọi nhiều nhất theo cột calls, không theo cột phần trăm thời gian.

System call	Calls	Vai trò trong ứng dụng
clock_nanosleep	301	Sensor chờ các mốc tuyệt đối; một chu kỳ có thể chia nhiều đoạn ngủ
rt_sigtimedwait	299	Main kiểm tra tín hiệu và thời lượng mỗi khoảng 100 ms
mq_timedreceive	225	Alert và Logger chờ dữ liệu từ hai queue
mq_timedsend	224	Gửi alarm, log và thông điệp STOP
futex	216	Đồng bộ mutex, condition variable và hoạt động pthread liên quan

Tổng cộng có 1609 lời gọi. mq_timedreceive chiếm 35,23% thời gian syscall, rt_sigtimedwait 20,39%, futex 19,30%, clock_nanosleep 19,22%. Những lời gọi chờ chiếm ưu thế là phù hợp với ứng dụng có tốc độ mẫu thấp và các worker blocking.

Bảng ghi 299 lỗi ở rt_sigtimedwait, phù hợp với timeout EAGAIN mà Main đã xử lý trong mã. Tuy nhiên bảng -c không chứa errno của từng lần gọi nên đây là diễn giải dựa trên mã, không phải xác nhận riêng từng lỗi. fatal_error của ứng dụng vẫn bằng 0.

Vì chạy MOCK, các read trong bảng này không phải read từ /dev/sms_sensor. Không thể dùng bảng này để ước lượng số lượt retry EAGAIN trên driver ARM. Phần trăm thời gian trong strace là phân bố thời gian syscall đã đo, không phải tỉ lệ CPU user/kernel hoặc phần trăm của toàn bộ 30 giây wall-clock.

Minh chứng: logs/strace_summary.txt; strace_app_output.txt; strace_exit_code.txt. Phiên trước rà soát cuối.

15 Phân tích CPU bằng perf

Perf record dùng sự kiện phần mềm cpu-clock ở tần số yêu cầu 999 Hz và call graph DWARF trên HOST/MOCK/OTHER trong 30 giây. Phiên thu được 59 mẫu, Lost Samples=0. Số mẫu ít do ứng dụng chủ yếu ngủ và chờ, nên tỷ lệ hotspot có độ phân giải thô.

Symbol trong báo cáo phẳng	Overhead
libc __syscall_cancel_arch_end	11.86%
kernel _raw_spin_unlock_irqrestore	8.47%
kernel finish_task_switch.isra.0	6.78%
kernel get_timespec64	5.08%
libc __syscall_cancel	5.08%

Symbol đứng đầu là wrapper syscall của libc, không phải rolling average. Call graph cho thấy các đường liên quan tới chờ tín hiệu, queue và đánh thức thread. Đây là điểm đáng chú ý: với ít phép tính trên mỗi mẫu, chi phí điều phối có thể nổi bật trong phần CPU đã sử dụng. Không suy ra một hàm tự viết chiếm chính xác cùng tỷ lệ.

Perf stat của phiên riêng	Giá trị
Wall-clock	30.104841743 s
Task-clock	40.49 ms
Context switches / CPU migrations	843 / 51
Page faults	82
User CPU / System CPU	0.010496 s / 0.053796 s

Trong tổng user+sys 0,064292 giây, user chiếm khoảng 16,33% và sys 83,67%. Đây là cơ cấu của CPU time được báo, không phải CPU bận suốt 83,67% thời gian chạy. Task-clock là số đo được báo riêng; không ép các trường kế toán khác phạm vi phải bằng nhau. Kết hợp các bằng chứng, workload này thiên về chờ sự kiện và I/O hơn CPU-bound.

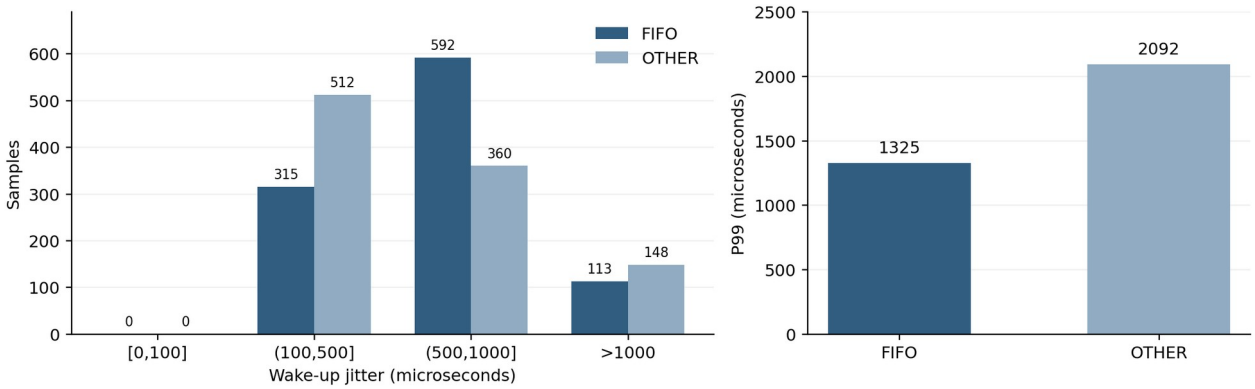
Một cải tiến có cơ sở là giảm lần đánh thức chỉ để kiểm tra stop bằng cơ chế chờ có thể đánh thức trực tiếp. Ngoài ra, 148 lseek trong strace gợi ý giảm ftello khi kiểm tra rotation, chẳng hạn duy trì bộ đếm byte tại Logger. Đây là đề xuất cần đo lại để xác nhận lợi ích, chưa phải tối ưu đã được chứng minh.

Minh chứng: logs/perf_report_flat.txt; perf_report.txt; perf_stat.txt. Các phiên trước rà soát cuối.

16 Kết quả so sánh jitter

Hai lần chạy trong cùng cấu hình QEMU dùng DRIVER, -i 500 và -d 510. Mỗi lần lưu 1020 độ trễ thức dậy vào RAM rồi xuất CSV sau join. FIFO dùng priority 50, OTHER priority 0; cả hai xác nhận mlockall OK. Hai lần chạy được thực hiện trước khi bổ sung log Sensor và sửa phép so sánh spike ở đợt rà soát cuối.

Chỉ số đơn vị µs	FIFO	OTHER
Số mẫu	1020	1020
Min	107	147
Mean	654.801	650.691
Max quan sát	1597	2598
P99 nearest rank	1325	2092
Mẫu âm	0	0



Hình 7 Histogram theo khoảng và P99 của hai phiên jitter

P99 lấy phần tử thứ ceil(0,99 × 1020)=1010 trong dãy tăng dần. Mức giảm P99 bằng (2092−1325)/2092 ×100 = 36,66%. Mean FIFO cao hơn khoảng 0,63%, vì vậy không thể kết luận FIFO tốt hơn ở mọi chỉ số. Max quan sát của FIFO thấp hơn và số lần trên 1 ms cũng ít hơn trong cặp phiên này.

Minh chứng: logs/jitter/sms_fifo.log.jitter.csv; sms_other.log.jitter.csv; jitter_summary.txt.

17 Đánh giá thời gian thực và cải tiến

Khoảng jitter µs	FIFO	OTHER
[0,100]	0	0
(100,500]	315	512
(500,1000]	592	360
>1000	113	148

Mẫu trên 1 ms chiếm khoảng 11,08% ở FIFO và 14,51% ở OTHER. Độ trễ đang đo là actual−expected trước thao tác read; chưa bao gồm retry, xử lý, message queue và ghi log. Hai phiên dài có 1018 và 1019 mẫu đọc thành công, mỗi phiên một deferred; số lần thức dậy vì vậy khác số mẫu cảm biến.

Ngưỡng chấp nhận trong phạm vi thiết kế

Với ứng dụng giám sát chậm có chu kỳ 500 ms, có thể chọn ngân sách thức dậy 5 ms tương đương 1% chu kỳ như một giả định thiết kế ban đầu. Cả max FIFO 1,597 ms và OTHER 2,598 ms quan sát được đều dưới ngưỡng giả định này. 5 ms không phải chuẩn chung của cảm biến công nghiệp; yêu cầu thực phải dựa trên quá trình được giám sát và deadline đầu-cuối, đặc biệt với ứng dụng điều khiển hoặc an toàn.

QEMU chạy trên HOST có scheduler riêng, và các phiên được chạy tuần tự với dữ liệu ngẫu nhiên. Chưa có nhiều lần lặp, kiểm soát tải hay thử nghiệm phần cứng ARM thật. Bản cuối có phiên kiểm tra ngắn 119 mẫu; bảng 1000+ mẫu không được gắn nhầm thành phép đo của chính binary sau sửa. Max hữu hạn không chứng minh worst-case hữu hạn.

Hướng cải tiến

Trước hết cần đo độ trễ từ deadline tới lúc read thành công và alarm được xử lý, đồng thời xem lại tương tác giữa lịch cố định và minimum interval của driver. Có thể phát triển driver có mẫu sẵn theo timer và cơ chế poll/wakeup, rồi so sánh với việc retry định kỳ hiện tại.

PREEMPT_RT tăng khả năng preempt trong kernel và dùng các cơ chế khóa hỗ trợ priority inheritance [5]. CPU affinity/isolation và interrupt affinity là các thử nghiệm tiếp theo để giảm nhiễu. Trong userspace có thể xem xét mutex priority inheritance và tránh chờ gửi log trên đường Sensor. Mọi thay đổi cần đo lại trong cùng workload; project chưa có kết quả PREEMPT_RT thực nghiệm.

18 Nhìn lại quá trình thực hiện

Điểm cần hiểu rõ khi ghép toàn hệ thống

Quá trình thực hiện cho thấy khó khăn không chỉ nằm ở một API mà ở việc phân biệt môi trường và ý nghĩa số liệu. Lệnh build chạy trên Ubuntu HOST, lệnh insmod chạy trong Linux ARM của QEMU, còn lệnh breakpoint chạy trong GDB. Khi nhầm môi trường, một lệnh đúng vẫn có thể thất bại. Việc ghi rõ dấu nhắc và lưu log từng phiên giúp kiểm tra lại thay vì dựa vào trí nhớ hoặc một ảnh chụp đơn lẻ.

Một bài học cụ thể là phân biệt lỗi và trạng thái chưa sẵn sàng. EAGAIN không đồng nghĩa driver hỏng; nó thể hiện hợp đồng minimum interval. Tăng số lần retry có thể giúp lấy đủ mẫu hơn nhưng tạo thêm hoạt động chờ. Từ đó, đánh giá hệ thống cần đồng thời xem sample count, retry count, jitter và luồng cảnh báo.

Điều sẽ thay đổi nếu thiết kế lại

Em sẽ xác định trước hợp đồng dữ liệu và thời gian: timestamp là thời điểm nào, Sensor được phép trễ bao lâu, buffer đầy thì bỏ dữ liệu nào và hệ thống báo lỗi ra sao. Em cũng sẽ thêm kiểm tra ngưỡng ngay từ đầu. Lỗi 15,2°C sang 20,2°C cho thấy so sánh float ở ranh giới có thể tạo sai lệch chức năng dù chương trình không crash.

Về timing, em sẽ ưu tiên tách thời điểm tạo mẫu khỏi thời điểm đọc và giảm polling retry. Về kiểm chứng, mỗi lần build dùng cho một phép đo cần được gắn với hash binary, cấu hình kernel, lệnh chạy và dữ liệu gốc. Điều này làm việc giải thích kết quả rõ ràng hơn khi mã tiếp tục được sửa.

Kỹ năng có thể áp dụng tiếp

Các kỹ năng có tính sử dụng lâu dài là đọc giao diện kernel/userspace, quản lý quyền sở hữu tài nguyên, phân tích lock order và tái hiện lỗi bằng test nhỏ. GDB giúp theo dõi đường đi dữ liệu; Valgrind kiểm tra việc cấp phát và giải phóng; strace/perf giúp chọn nơi cần phân tích thay vì tối ưu theo cảm giác.

Một hướng phát triển cụ thể là thêm luồng xuất dữ liệu ra mạng qua MQTT sau Processor, với queue có giới hạn và chính sách mất kết nối rõ ràng. Luồng mạng không nên chặn Sensor. Trước khi thêm giao diện hiển thị, cần hoàn thiện phép đo latency đầu-cuối và thử nghiệm tải kéo dài để hiểu giới hạn hệ thống.

19 Tái hiện và đối chiếu minh chứng

```
make -C app host debug test
make -C app arm
make -C driver arm KERNEL_DIR="$HOME/embedded_lab1/kernel/linux-5.15"
bash scripts/run_gdb_review.sh

# Trong QEMU sau khi cài app và đóng gói initramfs
insmod /lib/modules/$(uname -r)/sms_sensor_driver.ko
sms_app --fifo -i 500 -d 60 --log /tmp/sms.log
cat /proc/sms_stats
```

Khi nộp cần kèm mã nguồn, header, Makefile, binary ARM và HOST, initramfs và log theo cấu trúc đề bài. File PDF này trình bày thiết kế và bằng chứng; video demo 3–5 phút minh họa boot, device/procfs, verbose sample, cảnh báo, file log và Ctrl+C. Nguồn dữ liệu gốc cần được giữ cùng tên phiên để người đọc có thể kiểm tra lại.

Nội dung	Tệp để đối chiếu
ARM bản cuối	logs/final_review/qemu/sms_final_output.txt và sms_final.log
Driver và RAM	Cùng thư mục: sms_final_stats_*.txt, sms_final_memory_*s.txt, sms_final_maps_5s.txt
Valgrind và SIGINT	logs/final_review/valgrind_full.txt, valgrind_app_output.txt, valgrind_exit_code.txt
GDB	logs/final_review/gdb_session.txt
Jitter dài	logs/jitter/*.csv, jitter_summary.txt và sms_*_output.txt
Profiling	logs/strace_summary.txt, perf_report_flat.txt và perf_stat.txt

Tài liệu tham khảo

[1] Đề Assignment 02 Embedded Linux Systems và yêu cầu nộp bài được cung cấp trong học phần.

[2] Linux man pages. pthread_cond_wait và condition variables. https://man7.org/linux/man-pages/man3/pthread_cond_wait.3.html

[3] Linux man pages. clock_nanosleep. https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html

[4] Linux man pages. POSIX message queues. https://man7.org/linux/man-pages/man7/mq_overview.7.html

[5] Linux Kernel Documentation. Real time theory of operation. https://docs.kernel.org/core-api/real-time/theory.html

[6] Mã nguồn và log thực nghiệm project của Phạm Trần Gia Long SE201095, các gói Review và FinalEvidence, tháng 10 năm 2026.